COLLECTION GÉNIE LOGICIEL · LIVRE 1

Python

par la pratique

Partie 2 : faire des choix

et répéter des actions

Génie logiciel DQP / DTS

Option développement d'application

Moudériko Mouansié

Formateur référent en développement d'applications

MINEFOP Cameroun

Édition ASSIIN

Association des Informaticiens Innovants

Première version · Partie 2

Python par la pratique

Livre 1, Partie 2 : faire des choix et répéter des actions (conditions, boucles, chaînes de caractères)

Auteur : **Moudériko Mouansié**
Formateur référent en développement d'applications, MINEFOP Cameroun

Édition : **ASSIIN** (Association des Informaticiens Innovants)

Public : étudiants en Génie logiciel DQP / DTS, option développement d'application

Version de référence : **Python 3.15**. Première version, contenant les chapitres 5 à 7, trois travaux pratiques, un lexique et une fiche mémo.

© Moudériko Mouansié, Édition ASSIIN. Tous droits réservés. Toute reproduction, même partielle, est soumise à l'autorisation de l'auteur.

Sommaire

Avant-propos de la Partie 2 4

Partie 2 : Faire des choix et répéter des actions 5

Chapitre 5 : Les conditions 6

Chapitre 6 : Les boucles 12

Chapitre 7 : Les chaînes de caractères 17

Travaux pratiques de la Partie 2 23

TP 4 : Le jeu du nombre mystère 24

TP 5 : Tables de multiplication et motifs en étoiles 26

TP 6 : Un vérificateur de mot de passe 28

Lexique et fiche mémo de la Partie 2 30

Avant-propos de la Partie 2

Ce volume est la **deuxième partie** du Livre 1, *Python par la pratique*, destiné aux étudiants de Génie logiciel (DQP et DTS), option développement d'application. Il peut se lire seul, à condition d'avoir assimilé la Partie 1.

Ce que vous devez déjà savoir

- ouvrir l'invite de commandes (cmd), lancer l'interpréteur Python et exécuter un script ;
- créer des variables et connaître les types int, float, str et bool ;
- utiliser les opérateurs, `input()`, `print()` et les f-strings.

Ce que vous allez apprendre

Jusqu'ici, vos programmes exécutaient leurs instructions dans l'ordre, une seule fois. Dans cette partie, vous apprenez à **décider** (les conditions, chapitre 5), à **répéter** (les boucles, chapitre 6) et à **manipuler du texte** (les chaînes de caractères, chapitre 7). Avec ces outils, vos programmes deviennent réellement utiles.

Comment travailler

La méthode est la même que dans la Partie 1 : lisez le chapitre, reprenez chaque exemple dans l'interpréteur, expérimentez, puis faites les exercices et répondez aux questions « Testez-vous ». Les **exercices ne sont pas corrigés** (les corrigés sont remis par le formateur) : la rubrique « Pour vérifier » vous donne le résultat attendu quand c'est utile. En fin de partie, trois travaux pratiques réunissent toutes les notions.

> **Rappel des conventions**
> • `>>>` : invite de l'interpréteur (ne la tapez pas).
> • `...` : invite de continuation, qui apparaît quand vous tapez un bloc indenté ; validez par une ligne vide.
> • Les blocs de plus de quelques lignes sont à écrire dans un script `.py` lancé avec `python nom.py`.

> **Cette version**
> Ce volume contient les chapitres 5 à 7, trois travaux pratiques, un lexique et une fiche mémo. Il s'inscrit dans le plan général du Livre 1 : Partie 1 (chapitres 1 à 4), **Partie 2 (chapitres 5 à 7)**, puis les parties suivantes.

Partie 2
Faire des choix et répéter des actions

Chapitre 5 : Les conditions

Objectifs du chapitre

- faire exécuter du code seulement si une condition est vraie ;
- écrire des blocs `if`, `elif` et `else` correctement indentés ;
- combiner plusieurs conditions ;
- utiliser l'expression conditionnelle et l'instruction `match`.

5.1 Prendre une décision avec if

Jusqu'ici, vos programmes exécutaient toutes leurs instructions, dans l'ordre. Mais un programme utile doit pouvoir **décider** : afficher « majeur » ou « mineur » selon un âge, refuser une saisie incorrecte... L'instruction `if` (« si ») exécute un bloc de code seulement quand une condition est vraie.

Dans l'interpréteur Python

```
>>> age = 20
>>> if age >= 18:
...     print("Vous êtes majeur.")
...
Vous êtes majeur.
```

Trois éléments sont indispensables : la **condition** (ici `age >= 18`, dont le résultat est un booléen), le **deux-points** en fin de ligne, et le **bloc indenté** qui suit. Si la condition est fausse, le bloc est simplement ignoré.

L'indentation, une règle fondamentale

En Python, ce sont les **espaces au début des lignes** (l'indentation) qui délimitent un bloc. La convention est de **4 espaces**. Toutes les lignes d'un même bloc doivent avoir exactement le même décalage. Dans l'interpréteur, après une ligne se terminant par deux-points, l'invite devient `...` : tapez votre bloc, puis appuyez deux fois sur Entrée pour l'exécuter. Pour un programme de plus de quelques lignes, écrivez plutôt un script.

Voici les deux erreurs les plus fréquentes :

Deux-points oublié, puis indentation oubliée

```
>>> if age >= 18
  File "<python-input-1>", line 1
    if age >= 18
                ^
SyntaxError: expected ':'

>>> if age >= 18:
... print("Majeur")
  File "<python-input-2>", line 2
    print("Majeur")
    ^
IndentationError: expected an indented block after 'if' statement on line 1
```

5.2 Choisir entre deux voies : else

Avec `else` (« sinon »), on prévoit aussi le cas où la condition est fausse :

Dans l'interpréteur Python

```
>>> age = 16
>>> if age >= 18:
...     print("Majeur")
... else:
...     print("Mineur")
...
Mineur
```

Dans un script, la condition porte souvent sur une saisie de l'utilisateur :

majorite.py

```
# majorite.py
age = int(input("Votre âge : "))
if age >= 18:
    print("Vous êtes majeur.")
else:
    print("Vous êtes mineur.")
```

Exécution

```
C:\Users\etudiant\python_cours> python majorite.py
Votre âge : 20
Vous êtes majeur.
```

5.3 Plusieurs cas : elif

Quand il y a plus de deux possibilités, on ajoute autant de branches `elif` (contraction de « else if ») que nécessaire. Python teste les conditions **de haut en bas** et exécute uniquement la première branche dont la condition est vraie ; les autres sont ignorées.

Dans l'interpréteur Python

```
>>> moyenne = 13.5
>>> if moyenne >= 16:
...     mention = "Très bien"
... elif moyenne >= 14:
...     mention = "Bien"
... elif moyenne >= 12:
...     mention = "Assez bien"
... elif moyenne >= 10:
...     mention = "Passable"
... else:
...     mention = "Ajourné"
...
>>> print(mention)
Assez bien
```

Points d'attention

• L'**ordre** des branches compte. Si vous testiez `moyenne >= 10` en premier, une moyenne de 17 serait déclarée « Passable » : la première condition vraie gagne.

• Une seule des branches `if` / `elif` / `else` s'exécute. Pour tester des choses indépendantes, utilisez plusieurs `if` séparés.

5.4 Conditions composées

On combine les conditions avec `and`, `or` et `not` (chapitre 4). Python autorise aussi l'écriture mathématique d'un **intervalle**, et l'opérateur `in` teste si un morceau de texte est contenu dans un autre :

Dans l'interpréteur Python

```
>>> note = 11
>>> 10 <= note < 12
True
>>> note >= 10 and note < 12
True
>>> nom = "Mballa"
>>> "ball" in nom
True
>>> "z" in nom
False
>>> "z" not in nom
True
```

Voici un exemple complet : un tarif de cinéma qui dépend de l'âge.

Dans l'interpréteur Python

```
>>> age = 30
>>> est_etudiant = True
>>> if age < 12:
...     tarif = 2000
... elif age <= 25 or est_etudiant:
...     tarif = 3500
... else:
...     tarif = 5000
...
>>> tarif
3500
```

5.5 Valeurs « vraies » et « fausses »

Dans une condition, Python accepte n'importe quelle valeur, pas seulement un booléen. Sont considérées comme **fausses** : `0`, `0.0`, la chaîne vide `""` et `None`. Tout le reste est considéré comme vrai.

Dans l'interpréteur Python

```
>>> bool(0)
False
>>> bool(5)
True
>>> bool("")
False
>>> bool("a")
True
>>> bool(None)
False
```

Cela permet d'écrire des tests très lisibles, par exemple pour savoir si l'utilisateur a tapé quelque chose :

Dans l'interpréteur Python

```
>>> nom = ""
>>> if nom:
...     print("Nom saisi")
... else:
...     print("Nom vide")
...
Nom vide
```

5.6 L'expression conditionnelle

Quand une condition sert seulement à choisir entre deux valeurs, on peut tout écrire sur une ligne : `valeur_si_vrai if condition else valeur_si_faux`.

Dans l'interpréteur Python

```
>>> moyenne = 9.5
>>> resultat = "admis" if moyenne >= 10 else "ajourné"
>>> resultat
'ajourné'
```

Utilisez cette forme pour les cas simples ; dès que la logique se complique, préférez un vrai bloc `if`.

5.7 Pour aller plus loin : match et case

Depuis Python 3.10, l'instruction `match` permet de comparer une valeur à plusieurs cas, ce qui est plus lisible qu'une longue suite de `elif`. Le cas `_` joue le rôle de « tous les autres cas », et `|` regroupe plusieurs valeurs :

Dans l'interpréteur Python

```
>>> jour = 6
>>> match jour:
...     case 1:
...         print("Lundi")
...     case 2:
...         print("Mardi")
...     case 6 | 7:
...         print("Week-end")
...     case _:
...         print("Autre jour de la semaine")
...
Week-end
```

Pour en savoir plus, essayez `help("match")` dans l'interpréteur.

À retenir

- `if`, `elif` et `else` permettent d'exécuter des blocs différents selon des conditions.
- Chaque ligne de condition se termine par `:` et son bloc est indenté de 4 espaces.
- Python teste de haut en bas et exécute seulement la première branche vraie.
- On combine les conditions avec `and`, `or`, `not` et on teste l'appartenance avec `in`.
- 0, `""` et `None` sont faux ; le reste est vrai.
- `match` compare une valeur à plusieurs cas.

Exercices

Exercice 5.1 ★☆☆

Écrivez un script `age.py` qui demande un âge et affiche « majeur » ou « mineur ».

Pour vérifier : 18 donne majeur ; 17 donne mineur.

Exercice 5.2 ★☆☆

Écrivez un script qui demande un entier et dit s'il est pair ou impair (utilisez `%`).

Pour vérifier : 7 est impair ; 10 est pair.

Exercice 5.3 ★☆☆

Demandez deux nombres et affichez le plus grand. Traitez le cas où ils sont égaux.

Exercice 5.4 ★★☆

Écrivez un script `mention.py` qui demande une moyenne sur 20 et affiche la mention : « Très bien » à partir de 16, « Bien » à partir de 14, « Assez bien » à partir de 12, « Passable » à partir de 10, sinon « Ajourné ».

Pour vérifier : 16 donne Très bien ; 15.5 donne Bien ; 9.99 donne Ajourné.

Exercice 5.5 ★★☆

Écrivez un script qui dit si une année est bissextile : elle l'est si elle est divisible par 4 mais pas par 100, ou si elle est divisible par 400.

Pour vérifier : 2000 et 2024 sont bissextiles ; 1900 et 2026 ne le sont pas.

Exercice 5.6 ★★☆

Demandez trois longueurs. Dites si elles peuvent former un triangle (chaque côté doit être strictement plus petit que la somme des deux autres), puis, si c'est le cas, s'il est équilatéral, isocèle ou scalène.

Pour vérifier : 3, 3, 3 équilatéral ; 3, 3, 5 isocèle ; 3, 4, 5 scalène ; 1, 2, 5 impossible.

Exercice 5.7 ★★☆

Un cinéma applique trois tarifs : 2000 francs pour les moins de 12 ans, 3500 francs de 12 à 25 ans inclus, 5000 francs au-delà. Écrivez le script qui affiche le tarif selon l'âge saisi.

Pour vérifier : 8 donne 2000 ; 20 donne 3500 ; 40 donne 5000.

Exercice 5.8 ★★★

Résolvez l'équation $ax^2 + bx + c = 0$. Calculez le discriminant delta = b^2 - 4ac. S'il est négatif, il n'y a pas de solution réelle ; s'il est nul, il y a une solution double ; sinon, il y en a deux. Pour la racine carrée, utilisez `delta ** 0.5`.

Pour vérifier : (1, -3, 2) donne 2.0 et 1.0 ; (1, 2, 1) donne -1.0 ; (1, 0, 1) donne pas de solution réelle.

Exercice 5.9 ★★★

Écrivez une calculatrice : le script demande deux nombres puis un opérateur (`+`, `-`, `*` ou `/`), et affiche le résultat. Prévoyez un message pour une division par zéro et pour un opérateur inconnu (essayez avec `match`).

Pour vérifier : 8, 0, / doit afficher un message d'erreur clair.

Testez-vous

1. À quoi servent le deux-points et l'indentation après une ligne `if` ?
2. Pourquoi l'ordre des branches `elif` est-il important ?
3. Quelle différence entre plusieurs `if` successifs et une chaîne `if / elif / else` ?
4. Citez trois valeurs considérées comme fausses par Python.

Chapitre 6 : Les boucles

Objectifs du chapitre

- répéter des instructions avec `while` et `for` ;
- utiliser `range()` pour compter ;
- arrêter ou sauter une répétition avec `break` et `continue` ;
- imbriquer des boucles et éviter les boucles infinies.

6.1 Pourquoi répéter ?

Afficher cent fois un message, additionner les nombres de 1 à 1000, redemander une saisie tant qu'elle est incorrecte... Écrire cent fois la même instruction serait absurde. Une **boucle** répète un bloc d'instructions ; chaque passage s'appelle une **itération**. Python propose deux boucles : `while` (tant que une condition est vraie) et `for` (pour chaque élément d'une série).

6.2 La boucle while

La boucle `while` répète son bloc **tant que** sa condition est vraie. Elle demande toujours trois ingrédients : une variable **initialisée** avant la boucle, une **condition**, et une **mise à jour** de la variable dans le bloc.

Dans l'interpréteur Python

```
>>> compteur = 1
... while compteur <= 5:
...     print(compteur)
...     compteur += 1
...
1
2
3
4
5
>>> compteur
6
```

Une boucle peut aussi servir à **accumuler** un résultat, ici la somme des entiers de 1 à 10 :

Dans l'interpréteur Python

```
>>> somme = 0
... n = 1
... while n <= 10:
...     somme += n
...     n += 1
...
>>> somme
55
```

Points d'attention

• Si la condition ne devient jamais fausse, la boucle ne s'arrête jamais : c'est une **boucle infinie**. Ici, oublier `compteur += 1` bloquerait le programme. Pour l'interrompre, appuyez sur **Ctrl + C** dans l'invite de commandes.

Une utilisation très courante est la **validation de saisie** : on redemande tant que la valeur est incorrecte.

Validation d'une note

```
note = float(input("Note sur 20 : "))
while note < 0 or note > 20:
    print("La note doit être comprise entre 0 et 20.")
    note = float(input("Note sur 20 : "))
print("Note enregistrée :", note)
```

Exécution

```
Note sur 20 : 25
La note doit être comprise entre 0 et 20.
Note sur 20 : -3
La note doit être comprise entre 0 et 20.
Note sur 20 : 14
Note enregistrée : 14.0
```

6.3 La boucle for et la fonction range

La boucle `for` parcourt une série de valeurs, une à la fois. La fonction `range()` fabrique une série de nombres entiers : `range(5)` donne 0, 1, 2, 3, 4 ; `range(1, 6)` donne 1 à 5 ; `range(0, 10, 2)` donne 0, 2, 4, 6, 8 (le troisième nombre est le **pas**). La valeur de fin est toujours **exclue**.

Dans l'interpréteur Python

```
>>> for i in range(5):
...     print(i)
...
0
1
2
3
4
>>> list(range(1, 6))
[1, 2, 3, 4, 5]
>>> list(range(0, 10, 2))
[0, 2, 4, 6, 8]
>>> list(range(10, 0, -3))
[10, 7, 4, 1]
```

La fonction `list()` sera étudiée au chapitre 8 ; retenez pour l'instant qu'elle permet de voir les valeurs produites par `range()`.

Le mot qui suit `for` est une variable qui prend successivement chaque valeur. On peut aussi parcourir un texte lettre par lettre :

Dans l'interpréteur Python

```
>>> for lettre in "Python":
...     print(lettre)
...
P
y
t
h
o
n
>>> somme = 0
... for n in range(1, 11):
...     somme += n
...
>>> somme
55
```

for ou while ?

Utilisez **for** quand vous savez à l'avance combien de fois répéter, ou quand vous parcourez une série (nombres, lettres...). Utilisez **while** quand le nombre de répétitions dépend d'une condition qui change pendant l'exécution (saisie valide, jeu terminé...).

6.4 Interrompre une boucle : break et continue

L'instruction `break` **arrête** la boucle immédiatement. L'instruction `continue` **saute** le reste de l'itération et passe à la suivante.

Dans l'interpréteur Python

```
>>> for n in range(100, 200):
...     if n % 17 == 0:
...         print("Premier multiple de 17 :", n)
...         break
...
Premier multiple de 17 : 102
>>> for n in range(1, 10):
...     if n % 2 == 0:
...         continue
...     print(n)
...
1
3
5
7
9
```

Le motif `while True` (boucle volontairement infinie) combiné à `break` est très utile quand la décision d'arrêter se prend au milieu du bloc :

Un motif classique

```
while True:
    mot = input("Tapez stop pour arrêter : ")
    if mot == "stop":
        break
    print("Vous avez tapé :", mot)
print("Fin du programme")
```

6.5 Boucles imbriquées

Une boucle peut en contenir une autre. La boucle intérieure effectue tous ses tours **à chaque** tour de la boucle extérieure.

Dans l'interpréteur Python

```
>>> for ligne in range(1, 4):
...     for colonne in range(1, 4):
...         print(ligne * colonne, end=" ")
...     print()
...
1 2 3
2 4 6
3 6 9
>>> for i in range(1, 5):
...     print("*" * i)
...
*
**
***
****
```

Points d'attention

• Avec `range(1, 5)`, la dernière valeur est 4 et non 5 : c'est l'erreur du « décalage d'un » la plus fréquente. Vérifiez toujours les bornes.

• Ne modifiez pas la variable d'une boucle `for` dans son bloc : elle est réaffectée à chaque tour.

À retenir

• `while` répète tant qu'une condition est vraie : initialisation, condition, mise à jour.

• `for` parcourt une série ; `range(debut, fin, pas)` produit des entiers, fin exclue.

• `break` arrête la boucle, `continue` passe à l'itération suivante.

• Une boucle infinie s'arrête avec Ctrl + C.

• Les boucles imbriquées répètent la boucle intérieure à chaque tour de l'extérieure.

Exercices

Exercice 6.1 ★☆☆

Avec une boucle `while`, affichez les entiers de 1 à 10.

Exercice 6.2 ★☆☆
Avec une boucle `for` et `range()`, affichez les nombres pairs de 2 à 20.

Exercice 6.3 ★☆☆
Calculez la somme des entiers de 1 à 100.

Pour vérifier : vous devez obtenir 5050.

Exercice 6.4 ★☆☆
Demandez un nombre et affichez sa table de multiplication de 1 à 10.

Pour vérifier : pour 9, la ligne 7 est « 9 x 7 = 63 ».

Exercice 6.5 ★★☆
Calculez la factorielle d'un entier n saisi (n! = 1 × 2 × ... × n).

Pour vérifier : 5 donne 120 ; 0 donne 1 ; 10 donne 3628800.

Exercice 6.6 ★★☆
Comptez les voyelles (a, e, i, o, u) d'un mot saisi, sans tenir compte de la casse.

Pour vérifier : « Informatique » contient 6 voyelles.

Exercice 6.7 ★★☆
Demandez des notes jusqu'à ce que l'utilisateur tape -1. Affichez ensuite le nombre de notes saisies et leur moyenne.

Pour vérifier : 12, 15, 9, -1 donne 3 notes et une moyenne de 12.0.

Exercice 6.8 ★★☆
Affichez les 10 premiers termes de la suite de Fibonacci (chaque terme est la somme des deux précédents, en partant de 0 et 1).

Pour vérifier : 0, 1, 1, 2, 3, 5, 8, 13, 21, 34.

Exercice 6.9 ★★★
Demandez un entier et dites s'il est premier (divisible seulement par 1 et par lui-même).

Pour vérifier : 17 et 2 sont premiers ; 21 et 1 ne le sont pas.

Exercice 6.10 ★★★
Suite de Syracuse : partant d'un entier n, si n est pair on le divise par 2, sinon on le remplace par 3n + 1, jusqu'à atteindre 1. Comptez les étapes.

Pour vérifier : départ 6 donne 8 étapes ; départ 27 donne 111 étapes.

Testez-vous

1. Quels sont les trois ingrédients d'une boucle `while` ?
2. Que produit `range(2, 10, 3)` ?
3. Quelle différence entre `break` et `continue` ?
4. Comment interrompre un programme qui boucle à l'infini ?

Chapitre 7 : Les chaînes de caractères

Objectifs du chapitre

- accéder à un caractère ou à un morceau de texte avec les indices et les tranches ;
- utiliser les principales méthodes de chaînes ;
- comprendre qu'une chaîne ne se modifie pas ;
- nettoyer et vérifier des saisies textuelles.

7.1 Une chaîne est une série de caractères

Une chaîne est une série de caractères ordonnés, que l'on peut parcourir et dont on peut extraire des morceaux. Chaque caractère a une **position**, appelée **indice**, qui commence à **0**. Les indices négatifs comptent depuis la fin.

Caractère	P	y	t	h	o	n
Indice	0	1	2	3	4	5
Indice négatif	-6	-5	-4	-3	-2	-1

Dans l'interpréteur Python

```
>>> mot = "Python"
>>> mot[0]
'P'
>>> mot[5]
'n'
>>> mot[-1]
'n'
>>> mot[-2]
'o'
>>> len(mot)
6
```

Un indice hors limites

```
>>> mot[6]
Traceback (most recent call last):
  File "<python-input-9>", line 1, in <module>
    mot[6]
IndexError: string index out of range
```

Le dernier indice d'une chaîne de longueur `n` est `n - 1`.

7.2 Les tranches

Une **tranche** extrait un morceau de chaîne avec la notation `chaine[début:fin:pas]`. Le caractère de **fin** est exclu. Chaque valeur peut être omise : début par défaut au premier caractère, fin par défaut au dernier, pas par défaut 1.

Dans l'interpréteur Python

```
>>> mot = "Python"
>>> mot[0:2]
'Py'
>>> mot[2:]
'thon'
>>> mot[:3]
'Pyt'
>>> mot[-3:]
'hon'
>>> mot[::2]
'Pto'
>>> mot[::-1]
'nohtyP'
```

La tranche `[::-1]` parcourt le texte à l'envers : c'est le moyen le plus court d'inverser une chaîne.

7.3 Une chaîne ne se modifie pas

Les chaînes sont **immuables** : on ne peut pas changer un de leurs caractères sur place. Pour « modifier » un texte, on en construit un nouveau :

Une chaîne est immuable

```
>>> mot = "Python"
>>> mot[0] = "J"
Traceback (most recent call last):
  File "<python-input-11>", line 1, in <module>
    mot[0] = "J"
TypeError: 'str' object does not support item assignment
>>> "J" + mot[1:]
'Jython'
```

7.4 Parcourir une chaîne

On peut parcourir les lettres directement, ou passer par leurs indices avec `range(len(...))` quand on a besoin de la position :

Dans l'interpréteur Python

```
>>> mot = "abc"
>>> for lettre in mot:
...     print(lettre)
...
a
b
c
>>> for i in range(len(mot)):
...     print(i, mot[i])
...
0 a
1 b
2 c
```

7.5 Les méthodes de chaînes

Une **méthode** est une fonction attachée à une valeur, que l'on appelle avec un point : `valeur.méthode(arguments)`. Les chaînes en possèdent des dizaines. Toutes **renvoient une nouvelle chaîne** sans modifier l'originale.

Dans l'interpréteur Python

```
>>> texte = "  Bonjour le monde  "
>>> texte.strip()
'Bonjour le monde'
>>> texte.upper()
'  BONJOUR LE MONDE  '
>>> texte.lower()
'  bonjour le monde  '
>>> phrase = "bonjour le monde"
>>> phrase.capitalize()
'Bonjour le monde'
>>> phrase.title()
'Bonjour Le Monde'
>>> phrase.replace("le", "tout le")
'bonjour tout le monde'
>>> phrase.split()
['bonjour', 'le', 'monde']
>>> "-".join(phrase.split())
'bonjour-le-monde'
```

Dans l'interpréteur Python

```
>>> phrase = "bonjour le monde"
>>> phrase.find("le")
8
>>> phrase.find("xyz")
-1
>>> phrase.count("o")
3
>>> phrase.startswith("bon")
True
>>> phrase.endswith("x")
False
>>> "2026".isdigit()
True
>>> "abc".isalpha()
True
>>> "42".zfill(5)
'00042'
>>> "ab".center(10, "*")
'****ab****'
```

Méthode	Rôle
`upper()`, `lower()`	Tout en majuscules, tout en minuscules.
`capitalize()`, `title()`	Première lettre de la phrase, de chaque mot, en majuscule.
`strip()`	Supprime les espaces au début et à la fin.
`replace(a, b)`	Remplace chaque `a` par `b`.
`find(x)`, `count(x)`	Position de la première occurrence (-1 si absente), nombre d'occurrences.

Méthode	Rôle
`startswith(x)`, `endswith(x)`	Le texte commence ou finit-il par `x` ?
`split()`, `sép.join(liste)`	Découpe en mots ; assemble des mots avec un séparateur.
`isdigit()`, `isalpha()`	Que des chiffres ? que des lettres ?

Pour explorer les autres méthodes, tapez `help(str)` dans l'interpréteur.

Un réflexe utile pour traiter une saisie : **nettoyer** avant de comparer.

Dans l'interpréteur Python

```
>>> reponse = "  OUI "
>>> reponse.strip().lower() == "oui"
True
```

7.6 Textes sur plusieurs lignes et chaînes brutes

Entre **triples guillemets**, une chaîne peut s'étendre sur plusieurs lignes :

Chaîne multiligne

```
texte = """Première ligne
Deuxième ligne
Troisième ligne"""
print(texte)
```

Dans une chaîne normale, le caractère `\` introduit un caractère spécial (`\n`, `\t`), ce qui pose problème pour les chemins Windows. Une chaîne **brute**, précédée de `r`, désactive ce comportement :

Dans l'interpréteur Python

```
>>> print(r"C:\nouveau\test")
C:\nouveau\test
```

7.7 Les caractères et leur code

Chaque caractère possède un numéro dans la table Unicode. `ord()` donne le numéro d'un caractère et `chr()` fait l'inverse. Python 3.15 utilise l'**UTF-8** par défaut, ce qui permet de manipuler sans souci les accents et les alphabets du monde entier.

Dans l'interpréteur Python

```
>>> ord("A")
65
>>> ord("a")
97
>>> ord("é")
233
>>> chr(97)
'a'
>>> chr(ord("a") + 1)
'b'
```

À retenir

• Les caractères d'une chaîne ont un indice qui commence à 0 ; les indices négatifs comptent depuis la fin.
• `chaine[debut:fin:pas]` extrait une tranche (fin exclue) ; `[::-1]` inverse.
• Les chaînes sont immuables : les méthodes renvoient une nouvelle chaîne.
• `strip()`, `lower()`, `replace()`, `split()`, `find()` sont les méthodes les plus utiles.
• `in` teste si un texte est contenu dans un autre.
• Les triples guillemets créent des textes multilignes, le préfixe `r` des chaînes brutes.

Exercices

Exercice 7.1 ★☆☆
Avec le mot `"programmation"`, affichez sa première lettre, sa dernière lettre et sa longueur.

Pour vérifier : p, n et 13.

Exercice 7.2 ★☆☆
Avec le mot `"développement"`, affichez ses 4 premières lettres, ses 4 dernières lettres, puis une lettre sur deux.

Pour vérifier : « déve », « ment » et « dvlpeet ».

Exercice 7.3 ★☆☆
Demandez un mot et affichez-le à l'envers.

Pour vérifier : « python » donne « nohtyp ».

Exercice 7.4 ★★☆
Écrivez un script qui dit si un mot est un palindrome (il se lit de la même façon dans les deux sens), sans tenir compte de la casse.

Pour vérifier : « kayak » et « Radar » sont des palindromes ; « python » n'en est pas un.

Exercice 7.5 ★★☆
Demandez une phrase et affichez le nombre de mots qu'elle contient.

Pour vérifier : « Le Python c'est facile » contient 4 mots.

Exercice 7.6 ★★☆
Demandez un nom complet et affichez les initiales de chaque mot, séparées par des points.

Pour vérifier : « Jean Pierre Mballa » donne « J.P.M. ».

Exercice 7.7 ★★☆
Nettoyez un nom saisi de façon désordonnée : supprimez les espaces en trop et mettez une majuscule à chaque mot.

Pour vérifier : « mBALLA jean » donne « Mballa Jean ».

Exercice 7.8 ★★★
Écrivez un chiffrement de César : décalez chaque lettre minuscule de 3 rangs dans l'alphabet (le « z » revient au « c »). Aidez-vous de `ord()`, `chr()` et `%`.

Pour vérifier : « bonjour » donne « erqmrxu ».

Exercice 7.9 ★★★

Compressez une chaîne en indiquant chaque lettre suivie du nombre de répétitions consécutives.

Pour vérifier : « aaabbc » donne « a3b2c1 » ; « abc » donne « a1b1c1 ».

Exercice 7.10 ★★☆

Demandez un entier positif en redemandant tant que la saisie n'est pas composée uniquement de chiffres (utilisez `isdigit()`). Affichez ensuite le double de ce nombre.

Pour vérifier : « abc » est refusé ; « 21 » donne 42.

Testez-vous

1. Que renvoient `"Python"[0]` et `"Python"[-1]` ?
2. Que produit `"Python"[1:4]` ?
3. Pourquoi `mot[0] = "J"` provoque-t-il une erreur ?
4. Pourquoi appliquer `strip()` et `lower()` à une saisie avant de la comparer ?

Travaux pratiques de la Partie 2

Ces trois TP réunissent les conditions, les boucles et les chaînes de caractères, en plus des notions de la Partie 1. Même méthode que précédemment : un script par TP, lancé depuis l'invite de commandes, étape par étape, en comparant votre affichage à l'exemple fourni.

TP 4 : Le jeu du nombre mystère

Compétences mobilisées

Boucle `while`, conditions `if / elif / else`, saisie, conversion, validation avec `isdigit()`.

Durée estimée : 2 h.

Énoncé

L'ordinateur choisit un nombre secret entre 1 et 100 ; le joueur doit le deviner, avec au plus 7 essais. À chaque proposition, le programme répond « Trop petit ! » ou « Trop grand ! ». Écrivez le script **mystere.py**.

Tirer un nombre au hasard

Placez ces deux lignes au début du script : `import random` puis `secret = random.randint(1, 100)`. La fonction `randint(a, b)` renvoie un entier au hasard compris entre `a` et `b` inclus. Le mot `import` sera expliqué au chapitre 12 ; pour l'instant, utilisez-le tel quel. **Pour tester** votre programme avec l'exemple ci-dessous, remplacez temporairement la ligne du tirage par `secret = 42`.

Étape 1 : une partie sans limite

Redemandez une proposition tant que le nombre n'est pas trouvé, en comptant les essais. Affichez « Trop petit ! » ou « Trop grand ! » selon le cas, puis un message de victoire avec le nombre d'essais.

Étape 2 : limiter à 7 essais

Si le joueur n'a pas trouvé après 7 essais, affichez « Perdu ! Le nombre était ... ».

Étape 3 : valider la saisie

Si le joueur tape autre chose qu'un nombre entier, affichez « Veuillez entrer un nombre entier. » **sans compter** cet essai. Vérifiez la saisie avec `isdigit()` **avant** de la convertir avec `int()`.

Exemple d'exécution (avec secret = 42)

```
Essai 1/7 - votre proposition : 50
Trop grand !
Essai 2/7 - votre proposition : 25
Trop petit !
Essai 3/7 - votre proposition : abc
Veuillez entrer un nombre entier.
Essai 3/7 - votre proposition : 37
Trop petit !
Essai 4/7 - votre proposition : 43
Trop grand !
Essai 5/7 - votre proposition : 40
Trop petit !
Essai 6/7 - votre proposition : 42
Bravo ! Vous avez trouvé 42 en 6 essai(s).
```

Pour aller plus loin (facultatif)

- Proposez de rejouer à la fin d’une partie.
- Adaptez le nombre d’essais maximum à la taille de l’intervalle.
- Affichez un message différent si le joueur trouve en 3 essais ou moins.

Critères de réussite

- Le programme reproduit l’exemple pour les mêmes propositions.
- Une saisie invalide ne fait pas planter le programme et ne coûte pas d’essai.
- Le nombre d’essais maximum est stocké dans une constante.

TP 5 : Tables de multiplication et motifs en étoiles

Compétences mobilisées

Boucles `for` et `range()`, boucles imbriquées, répétition de chaînes, alignement avec les f-strings.

Durée estimée : 1 h 30.

Énoncé

Écrivez un script **motifs.py** qui réalise trois affichages, l'un après l'autre.

Étape 1 : une table de multiplication

Demandez un nombre entier et affichez sa table de multiplication de 1 à 10, avec le titre et le soulignement de l'exemple, et des colonnes bien alignées (le nombre sur 2 caractères, le multiplicateur sur 2, le résultat sur 3).

Exemple d'exécution

```
Nombre : 7

Table de multiplication de 7
----------------------------
 7 x  1 =   7
 7 x  2 =  14
 7 x  3 =  21
 7 x  4 =  28
 7 x  5 =  35
 7 x  6 =  42
 7 x  7 =  49
 7 x  8 =  56
 7 x  9 =  63
 7 x 10 =  70
```

Étape 2 : un triangle d'étoiles

Demandez une hauteur et affichez un triangle rectangle d'étoiles de cette hauteur.

Exemple d'exécution

```
Hauteur : 4

*
**
***
****
```

Étape 3 : une pyramide centrée

Avec la même hauteur, affichez une pyramide : la ligne 1 contient 1 étoile, la ligne 2 en contient 3, la ligne 3 en contient 5, etc., chaque ligne étant précédée du bon nombre d'espaces pour que la pyramide soit centrée.

Exemple d'exécution (hauteur 4)

```
   *
  ***
 *****
*******
```

Indices : pour la ligne numéro `i` (en partant de 1), il faut `2 * i - 1` étoiles et `hauteur - i` espaces ; multipliez une chaîne par un entier pour la répéter.

Pour aller plus loin (facultatif)

- Affichez la **table de Pythagore** complète de 1 à 9 sous forme de grille, avec des boucles imbriquées et un alignement sur 4 caractères par colonne.
- Affichez un losange en combinant la pyramide et sa version inversée.

Critères de réussite

- Les trois affichages sont identiques aux exemples pour les mêmes valeurs saisies.
- Le programme fonctionne pour d’autres valeurs (table de 12, hauteur 6).
- Vous n’avez écrit aucune ligne d’affichage en double : tout passe par des boucles.

TP 6 : Un vérificateur de mot de passe

Compétences mobilisées

Parcours de chaîne avec `for`, méthodes `isupper()`, `islower()`, `isdigit()`, `len()`, boucle `while`, booléens.

Durée estimée : 2 h.

Énoncé

Un site impose des règles pour les mots de passe. Écrivez le script **motdepasse.py** qui demande un mot de passe et le refuse tant qu'il ne respecte pas **toutes** les règles :

- au moins 8 caractères ;
- au moins une majuscule ;
- au moins une minuscule ;
- au moins un chiffre.

Étape 1 : tester une règle

Commencez par vérifier la longueur seule. Puis, pour la règle des majuscules, parcourez le mot de passe lettre par lettre avec une boucle `for` et utilisez une variable booléenne, initialisée à `False`, que vous passez à `True` dès qu'une majuscule est rencontrée (`isupper()`).

Étape 2 : toutes les règles

Appliquez la même idée aux minuscules et aux chiffres. Pour chaque règle non respectée, affichez une ligne « Refusé : ... » en suivant l'ordre de la liste ci-dessus.

Étape 3 : redemander

Placez le tout dans une boucle qui redemande un mot de passe tant qu'il est refusé, et comptez les tentatives pour les afficher à la fin.

Exemple d'exécution

```
Choisissez un mot de passe : abc
Refusé : il faut au moins 8 caractères.
Refusé : il faut au moins une majuscule.
Refusé : il faut au moins un chiffre.
Choisissez un mot de passe : motdepasse
Refusé : il faut au moins une majuscule.
Refusé : il faut au moins un chiffre.
Choisissez un mot de passe : Motdepasse1
Mot de passe accepté en 3 tentative(s).
```

Remarque de sécurité

Ce programme est un exercice : dans une vraie application, on ne stocke jamais un mot de passe en clair et on ne l'affiche pas à l'écran. N'utilisez jamais vos vrais mots de passe pour tester.

Pour aller plus loin (facultatif)

- Ajoutez une règle : au moins un caractère spécial (un caractère qui n’est ni lettre ni chiffre).
- Refusez les mots de passe contenant des espaces.
- Limitez à 5 tentatives.

Critères de réussite

- Le programme reproduit l’exemple pour les mêmes saisies.
- Tous les messages d’erreur applicables s’affichent en une seule fois, dans l’ordre.
- Le code est lisible : une variable booléenne par règle, des commentaires sur chaque bloc.

Lexique et fiche mémo de la Partie 2

Lexique

Terme	Définition
Condition	Expression dont le résultat est vrai ou faux, qui décide de l'exécution d'un bloc.
Bloc	Groupe d'instructions indentées qui dépendent d'une même ligne se terminant par deux-points.
Indentation	Décalage des lignes d'un bloc (4 espaces) ; il fait partie de la syntaxe de Python.
Branche	Un des chemins possibles d'une structure if / elif / else.
Boucle	Structure qui répète un bloc d'instructions.
Itération	Un passage dans le bloc d'une boucle.
Compteur	Variable qui compte les itérations ou les événements.
Accumulateur	Variable qui cumule un résultat (somme, produit...) au fil des itérations.
Boucle infinie	Boucle dont la condition ne devient jamais fausse (Ctrl + C pour l'arrêter).
Imbriquée	Placée à l'intérieur d'une autre structure (boucle dans une boucle, etc.).
Séquence	Série ordonnée d'éléments que l'on peut parcourir (une chaîne, un range...).
Indice	Position d'un élément dans une séquence, à partir de 0.
Tranche	Morceau de séquence extrait avec [début:fin:pas].
Méthode	Fonction attachée à une valeur, appelée avec valeur.méthode().
Immuable	Qui ne peut pas être modifié sur place (une chaîne, par exemple).

Fiche mémo

Les structures de la Partie 2

```
# Condition
if condition1:
    ...
elif condition2:
    ...
else:
    ...

# Boucles
while condition:
    ...
for variable in range(debut, fin, pas):
    ...            # break : arrêter ; continue : itération suivante

# Chaînes
mot[i]   mot[-1]   mot[a:b]   mot[::-1]   len(mot)
mot.strip()  mot.lower()  mot.replace(a, b)  mot.split()  mot.find(x)
```